Lab 8: OpenTelemetry for the Village Pond Planning System

Sneha Nagmoti
Indian Institute of Technology Bhilai snehan@iitbhilai.ac.in

16 October 2026

1 Overview

OpenTelemetry (OTel) was added to my Assignment 1 backend (FastAPI service `village-pond-api`). The service now emits all three signals over OTLP: **traces** (one per request, with a child span for every external data source and every terrain/hydrology stage), **metrics** (request counts, latency, per-stage time, catchment area, runoff volume, external-source failures) and **logs** (every Python log record, tagged with the trace and span that produced it). An OpenTelemetry Collector routes traces to **Jaeger**, metrics to **Prometheus**, and logs to a JSON file and its console. All screenshots below were taken from my own running project processing the supplied `contours_1m.kml`.

Code: https://github.com/snehanagmoti/Village-Pond-planning (commits `bacc3d2`, `2365037`; files `backend/observability.py`, `observability/`).

2 Implementation

2.1 Architecture

`FastAPI app (OTel SDK)` $\xrightarrow{\text{OTLP/HTTP :4318}}$ `OTel Collector` →
`traces` → Jaeger (UI :16686)
`metrics` → Prometheus exporter :8889 ← scraped by Prometheus (UI :9090)
`logs` → `logs.jsonl` file + debug console

Sending everything to a Collector (instead of directly to Jaeger/Prometheus) keeps the application vendor-neutral: changing a backend only changes `collector.yaml`, not the code.

2.2 What is instrumented

Setup (`backend/observability.py`). `setup_observability(app)` creates a `Resource` (`service.name=village-pond-api`, `service.version`, `deployment.environment`), a `TracerProvider` with a `BatchSpanProcessor`, a `MeterProvider` with a 5-second `PeriodicExportingMetricReader`, and a `LoggerProvider` with a `BatchLogRecordProcessor`, all using OTLP/HTTP exporters. Telemetry is *opt-in* (`OTEL_ENABLED=true`). When disabled, or if the packages are missing (e.g. on the older lab deployment), every helper is a no-op, so behaviour is unchanged. 87 backend tests pass, including a new test that checks the pipeline spans and metrics with an in-memory exporter.

Automatic instrumentation. `FastAPIInstrumentor` creates a server span per request (method, route, status code, client) and the standard `http.server.*` metrics. `HTTPXClientInstrumentor` creates a client span for every outgoing call to the elevation, imagery, map-tile and rainfall APIs, so failures of third-party services appear inside the request trace.

Manual spans (with a context-manager helper `span(name, **attributes)`):

Span	Attributes recorded
`external.satellite_imagery`, `external.mapped_hydrography`, `external.rainfall`, `external.elevation`	error status + exception when a source fails
`contour.parse_kml`	file size, file name, number of contour lines
`terrain.harmonic_interpolation`	grid rows/columns, cell size, iterations, converged
`water.exclusion_mask`	detected-water ratio, excluded cells (60 m buffer)
`terrain.priority_flood`	filled cells, maximum fill depth
`terrain.resolve_flats_d8`, `terrain.flow_accumulation`	maximum accumulation
`terrain.rank_candidates`	eligible cells, selection mode, options ranked
`terrain.delineate_catchment`	catchment cells and hectares
server span (result)	`pond.catchment_ha`, `pond.runoff_m3`, `pond.capacity_m3`, `pond.selection_mode`, `pond.analysis_status`, pond coordinates

The CPU-heavy pipeline runs in a worker thread (`asyncio.to_thread`); because that call copies the context, the stage spans are correctly nested under the request span.

Custom metrics: `pond.analysis.requests` (counter: workflow, HTTP status, outcome), `pond.analysis.duration` and `pond.pipeline.stage.duration` (histograms, seconds), `pond.catchment.area` (ha) and `pond.runoff.volume` (m^3) histograms, and `pond.external_source.failures` (counter: source, reason). Default OTel histogram buckets are designed for milliseconds; with them the p95 of a 1.3 s analysis was reported as 4.75 s, so explicit second-scale bucket boundaries were configured.

Logs: an OTel `LoggingHandler` on the root logger exports every record with its trace/span id; a logging filter also prints `[trace_id=... span_id=...]` on console lines. New log events record each completed analysis and every external-source failure.

2.3 Tools, libraries and backend services

Component	Version / role
`opentelemetry-api`, `-sdk`, `-exporter-otlp-proto-http`	1.45.1 — providers, processors, OTLP/HTTP exporters
`opentelemetry-instrumentation-fastapi`, `-httpx`	0.66b1 — automatic server and HTTP-client spans
OpenTelemetry Collector (contrib)	0.111.0 — OTLP receiver; Jaeger, Prometheus, file and debug exporters
Jaeger all-in-one	1.62.0 — trace storage and UI
Prometheus	2.53.2 — metric scraping, PromQL and UI
Application	FastAPI, Uvicorn, NumPy, OpenCV (Assignment 1 code)

2.4 How it was run

Two instances of the same app were started with `observability/run_app.py`. They are distinguished by the resource attribute `deployment.environment`:

- `lab-live-network` (port 8002) uses the real external adapters. My lab machine has no access to the Esri/OpenStreetMap tile servers or the rainfall and elevation APIs, so these requests show genuine failures.
- `lab-offline-fixtures` (port 8001) replaces *only* the three external data adapters (satellite tiles, OSM tiles, rainfall) with deterministic local fixtures: no water detected, and about 1,216 mm/year of rain. KML parsing, interpolation, Priority-Flood, D8, ranking, catchment and runoff code run unchanged on the real `contours_1m.kml`. Because the fixture detects no water, the catchment numbers (50.32 ha) differ slightly from the deployed site (50.90 ha).

Workload sent with `curl`: automatic, point and region contour analyses (HTTP 200); a point on the river, a missing upload, a wrong field name and a malformed KML (HTTP 422); a burst that hit the rate limit (HTTP 429); contour and live analyses on the network instance (HTTP 503); and health checks.

3 Screenshots and what they show

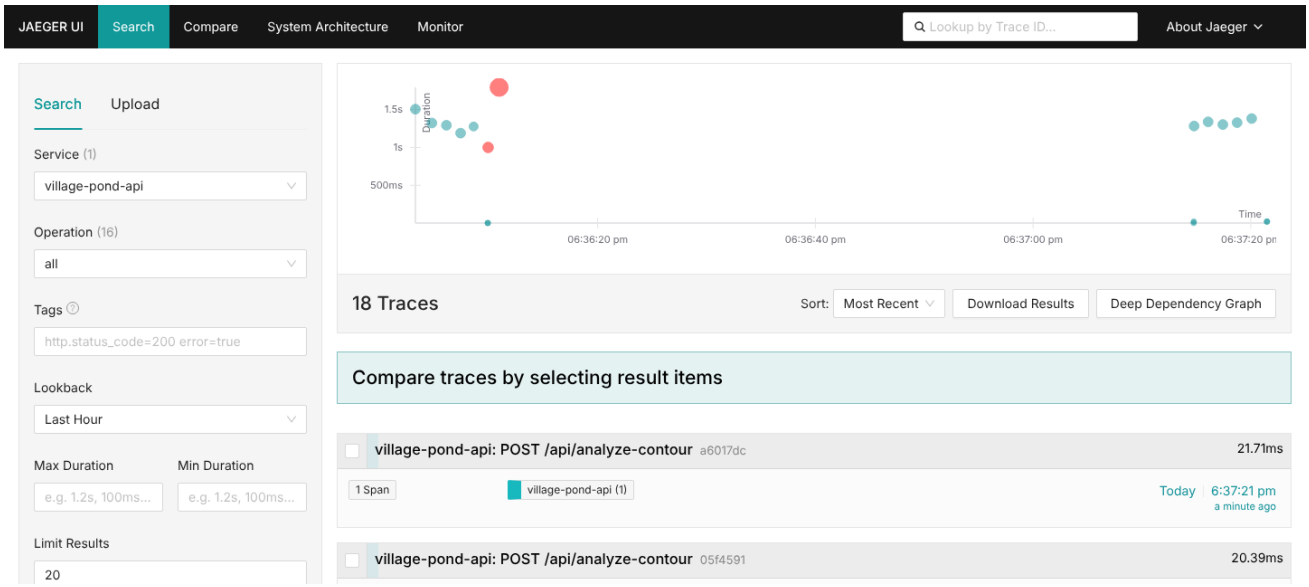


Figure 1: Jaeger search for service `village-pond-api`. Each dot/row is one request trace; the y-axis is duration. Successful contour analyses take about 1.3–1.5 s with 12 spans, rejected requests (422/429) take milliseconds with 1 span, and failed requests (503) show error markers.

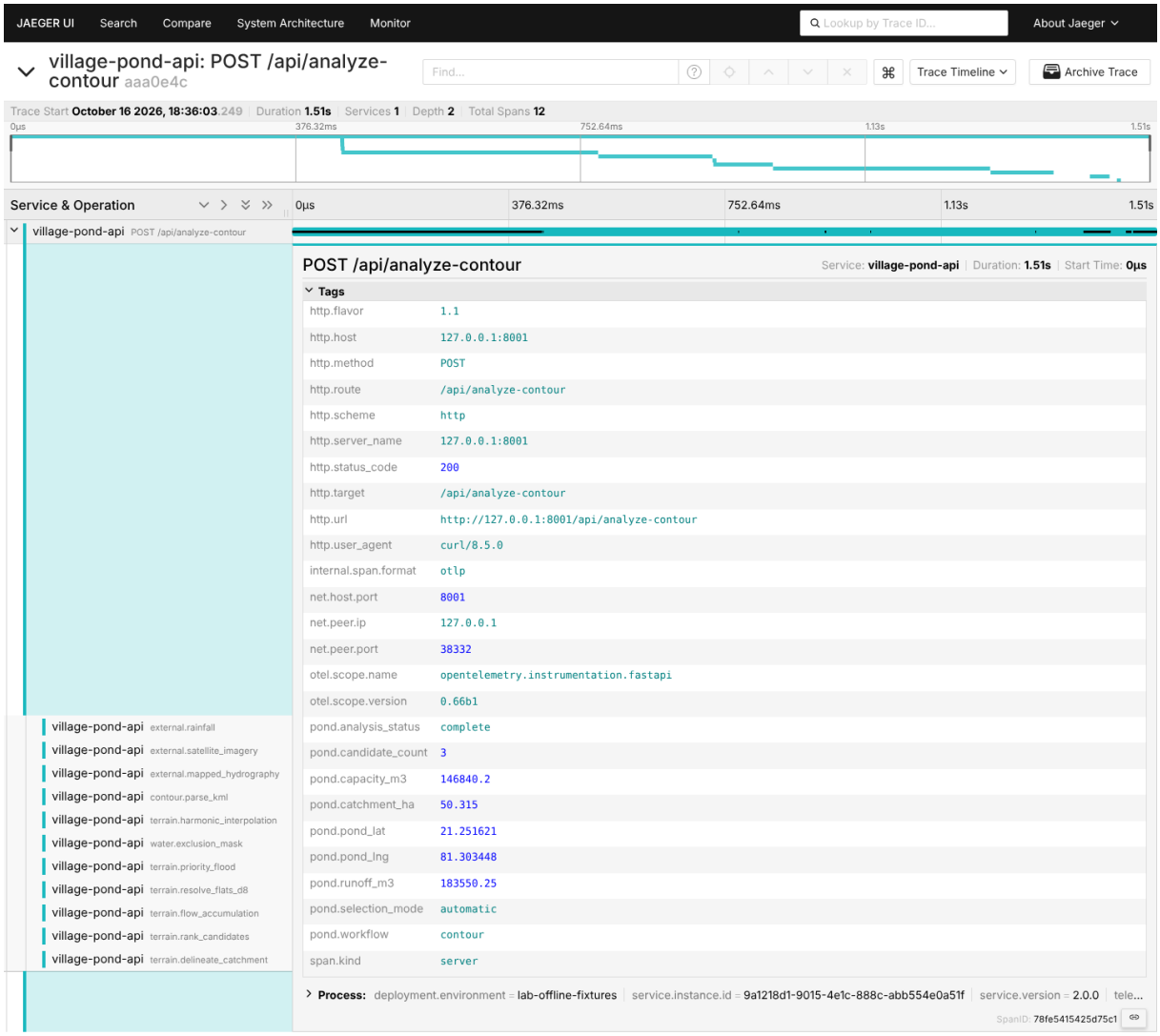


Figure 2: Trace of one successful automatic contour analysis (1.51 s, 12 spans). The waterfall shows the three external fetches running *in parallel* (asyncio.gather), then KML parsing (339 ms), harmonic interpolation (150 ms), water exclusion (1 ms), Priority-Flood (77 ms), flat resolution + D8 (286 ms), flow accumulation (63 ms), ranking (26 ms) and catchment delineation (3 ms). The server-span tags carry the scientific result: catchment 50.315 ha, runoff 183,550 m^3/year, capacity 146,840 m^3, pond at 21.251621, 81.303448, 3 candidates, mode `automatic`.

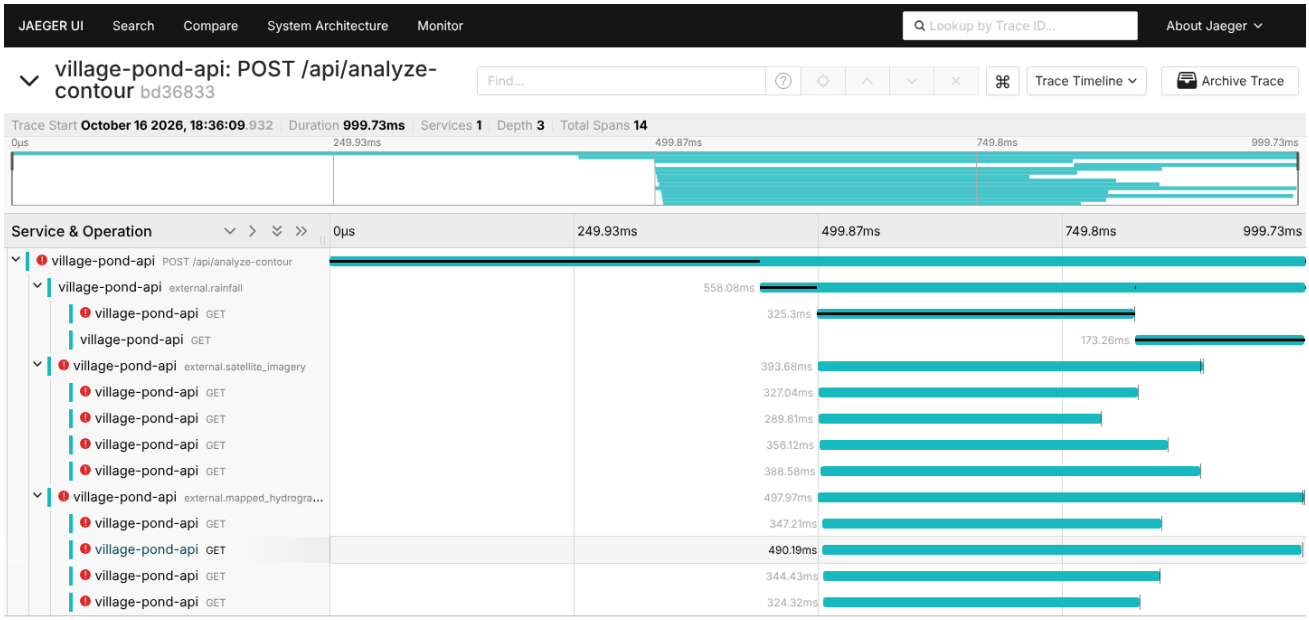


Figure 3: Trace of a contour request on the `lab-live-network` instance that returned HTTP 503. Red error icons mark the failed spans. Under `external.satellite_imagery` and `external.mapped_hydrography`, the auto-instrumented httpx `GET` spans show each tile download failing. The request fails closed: no pond is recommended without water screening.

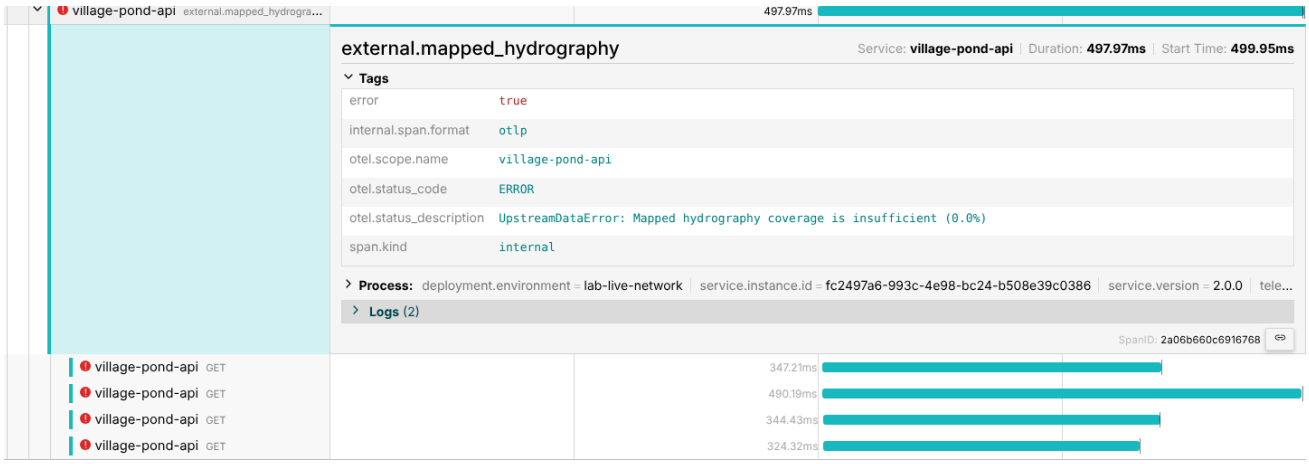


Figure 4: Detail of the failed `external.mapped_hydrography` span: `error=true`, `otel.status_code=ERROR` and the exact reason *"Mapped hydrography coverage is insufficient (0.0%)"*. The `Process` line shows the resource attributes identifying which deployment produced the span.

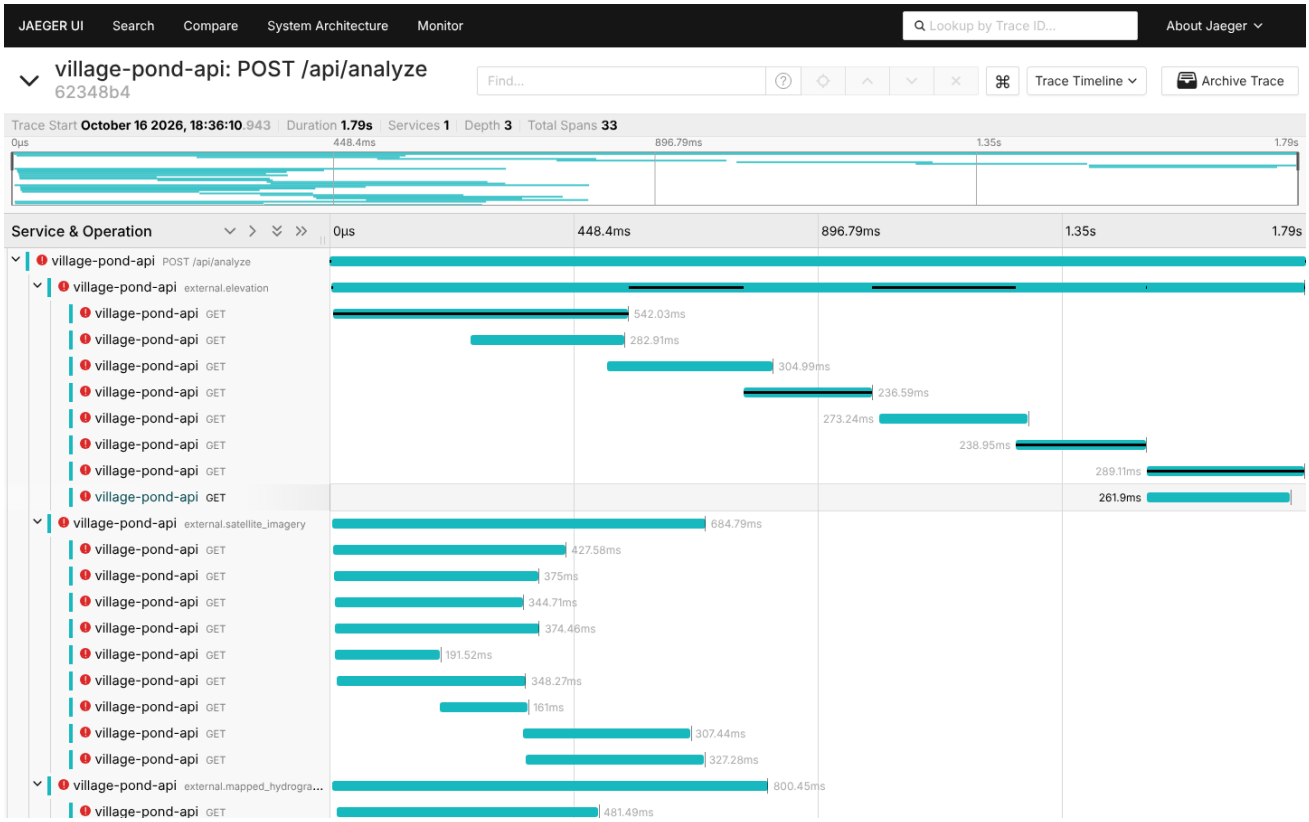


Figure 5: Live-map analysis (`POST /api/analyze`, 33 spans, 1.79 s) failing with 503. The elevation adapter tries six batch requests and then its tile fallback; every attempt is a separate red `GET` span. The trace shows where the 1.8 s went (retries against an unreachable service), which a single error message could not.

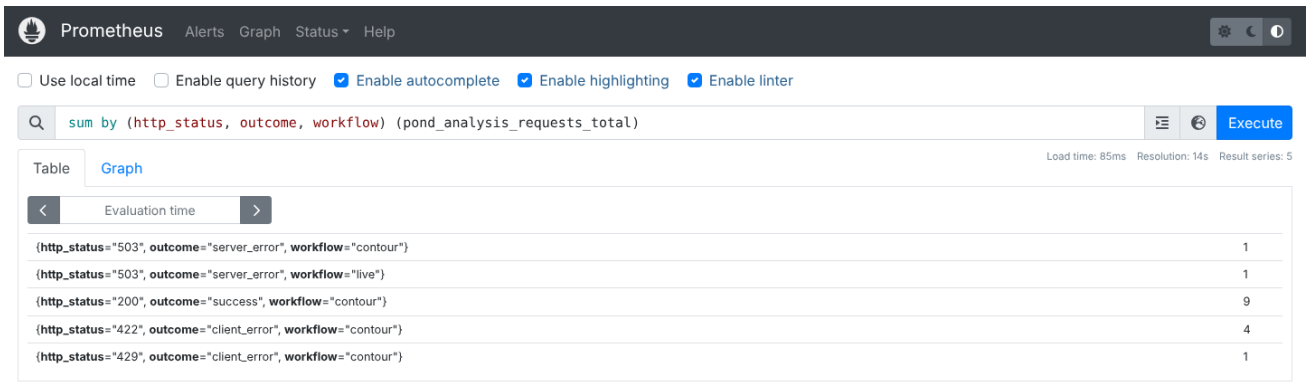


Figure 6: Prometheus: `pond.analysis.requests` by HTTP status, outcome and workflow (9 successes, 4 validation errors, 1 rate-limited request, 2 upstream failures). This is the error-rate view used for monitoring.

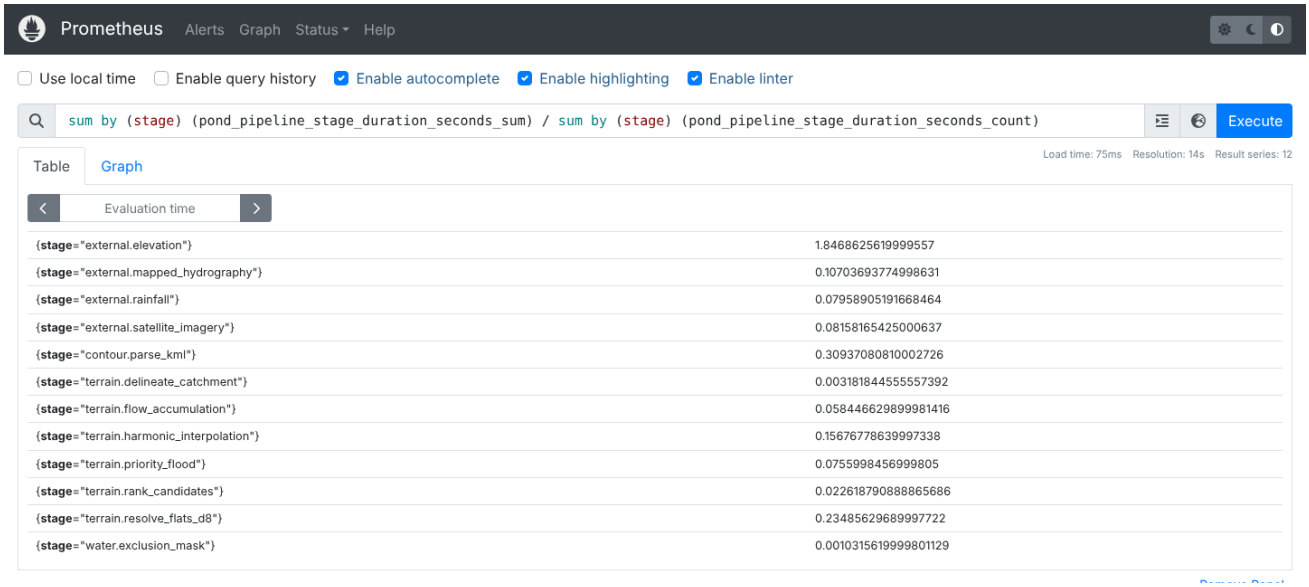


Figure 7: Average time per pipeline stage from the `pond.pipeline.stage.duration` histogram. KML parsing (0.32 s) and flat resolution + D8 (0.24 s) dominate; catchment delineation is about 3 ms. External elevation (1.8 s) appears only in the failing live workflow.

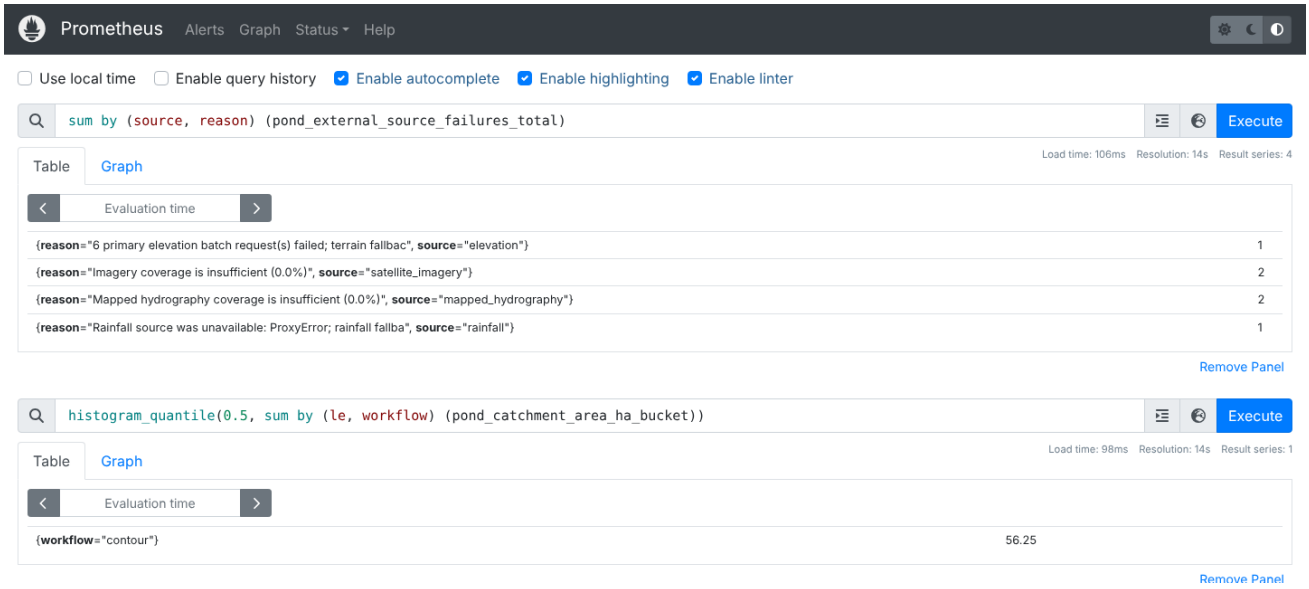


Figure 8: Top: `pond.external_source.failures` by source and reason (satellite imagery 2, mapped hydrography 2, rainfall 1, elevation 1). Bottom: median catchment area estimated from the histogram buckets (56 ha). Prometheus interpolates inside the 50–75 ha bucket; the exact values are in the traces.

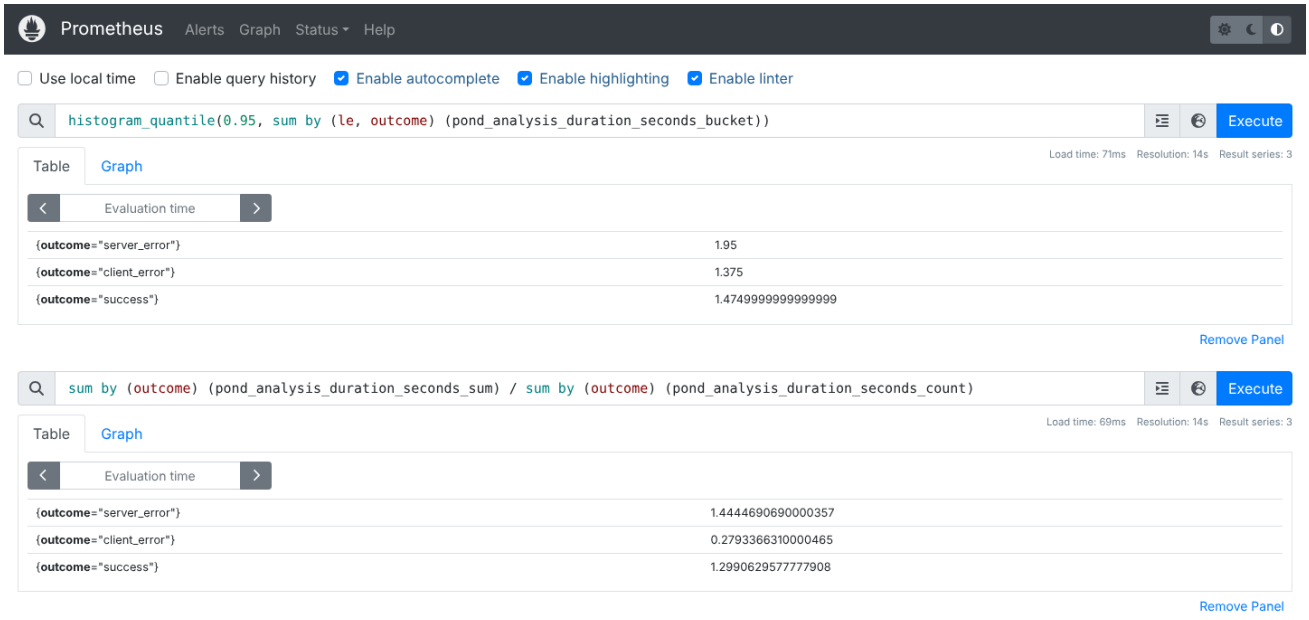


Figure 9: Latency of analysis requests: p95 from the histogram (success 1.47 s, client error 1.38 s, server error 1.95 s) and mean duration per outcome (success 1.30 s). The client-error p95 is high because a river-point request is rejected only *after* terrain analysis, while missing-file errors take milliseconds.

OpenTelemetry log records received by the Collector (file exporter: data/logs.jsonl)

Each Python log record exported over OTLP carries the trace_id/span_id of the request that produced it, plus resource attribute deployment.environment.

time	severity	logger (scope)	body	trace_id	span_id	deployment.environment
13:06:04.732	INFO	routers.pond_planner	contour_analysis_complete status=complete mode=automatic catchment_ha=50.32 runoff_m3=183550 candidates=3	aaa0e4c5f591ffba…	78fe5415425d75c1	lab-offline-fixtures
13:06:04.753	INFO	main	request_complete method=POST path=/api/analyze-contour status=200 duration_ms=1484.4	aaa0e4c5f591ffba…	78fe5415425d75c1	lab-offline-fixtures
13:06:06.072	INFO	routers.pond_planner	contour_analysis_complete status=complete mode=point catchment_ha=28.45 runoff_m3=103771 candidates=3	eb41ff2bc8e979db…	c6edd54fe5ac5308	lab-offline-fixtures
13:06:06.088	INFO	main	request_complete method=POST path=/api/analyze-contour status=200 duration_ms=1321.5	eb41ff2bc8e979db…	c6edd54fe5ac5308	lab-offline-fixtures
13:06:07.374	INFO	routers.pond_planner	contour_analysis_complete status=complete mode=point catchment_ha=37.91 runoff_m3=138283 candidates=3	847fbdb9be1455c0…	71dd34c6e50d223a	lab-offline-fixtures
13:06:07.390	INFO	main	request_complete method=POST path=/api/analyze-contour status=200 duration_ms=1291.8	847fbdb9be1455c0…	71dd34c6e50d223a	lab-offline-fixtures
13:06:08.571	INFO	routers.pond_planner	contour_analysis_complete status=complete mode=region catchment_ha=10.04 runoff_m3=36639 candidates=3	d25fe4b6fa8e7955…	d6b94238466bd584	lab-offline-fixtures
13:06:08.588	INFO	main	request_complete method=POST path=/api/analyze-contour status=200 duration_ms=1187.3	d25fe4b6fa8e7955…	d6b94238466bd584	lab-offline-fixtures
13:06:09.874	INFO	main	request_complete method=POST path=/api/analyze-contour status=422 duration_ms=1275.3	9d99b820fb14985d…	ee44ff297fa6cde1	lab-offline-fixtures
13:06:09.893	INFO	main	request_complete method=POST path=/api/analyze-contour status=422 duration_ms=1.1	8703c797057057a4…	ae0c9930f6f5b0b1	lab-offline-fixtures
13:06:09.906	INFO	main	request_complete method=GET path=/health/ready status=200 duration_ms=1.3	1c36d83a409c0310…	27adde6f7c113bad	lab-offline-fixtures
13:06:10.825	WARN	routers.pond_planner	external_source_failed source=external.satellite_imagery reason=Imagery coverage is insufficient (0.0%)	bd368331b4e7af3e…	d4e70e658a2c21ab	lab-live-network
13:06:10.928	WARN	routers.pond_planner	external_source_failed source=external.mapped_hydrography reason=Mapped hydrography coverage is insufficient (0.0%)	bd368331b4e7af3e…	2a06b660c6916768	lab-live-network
13:06:10.931	INFO	main	request_complete method=POST path=/api/analyze-contour status=503 duration_ms=982.7	bd368331b4e7af3e…	7b5fd53917e435e1	lab-live-network
13:06:11.599	WARN	services.rainfall	rainfall_fallback_failed error_type=UpstreamDataError	62348b4937a03e67…	6ec84e7416bcce42	lab-live-network
13:06:11.602	WARN	routers.pond_planner	external_source_failed source=external.rainfall reason=Rainfall source was unavailable: ProxyError; rainfall fallback was also unavailable	62348b4937a03e67…	6ec84e7416bcce42	lab-live-network
13:06:11.632	WARN	routers.pond_planner	external_source_failed source=external.satellite_imagery reason=Imagery coverage is insufficient (0.0%)	62348b4937a03e67…	e83ae9732819f5e6	lab-live-network
13:06:11.747	WARN	routers.pond_planner	external_source_failed source=external.mapped_hydrography reason=Mapped hydrography coverage is insufficient (0.0%)	62348b4937a03e67…	cab4f9693e0cdd18	lab-live-network
13:06:12.442	WARN	services.elevation	elevation_batch_failed error_type=ProxyError	62348b4937a03e67…	8f6c5389f3267583	lab-live-network
13:06:12.442	WARN	services.elevation	elevation_batch_failed error_type=ProxyError	62348b4937a03e67…	8f6c5389f3267583	lab-live-network
13:06:12.442	WARN	services.elevation	elevation_batch_failed error_type=ProxyError	62348b4937a03e67…	8f6c5389f3267583	lab-live-network
13:06:12.442	WARN	services.elevation	elevation_batch_failed error_type=ProxyError	62348b4937a03e67…	8f6c5389f3267583	lab-live-network

Figure 10: OTLP log records received by the Collector (file exporter). Each record has severity, logger, message, **trace_id/span_id** and `deployment.environment`. Successful analyses log mode, catchment and runoff; WARN rows record which external source failed and why. Any row can be matched to its Jaeger trace through the trace id.

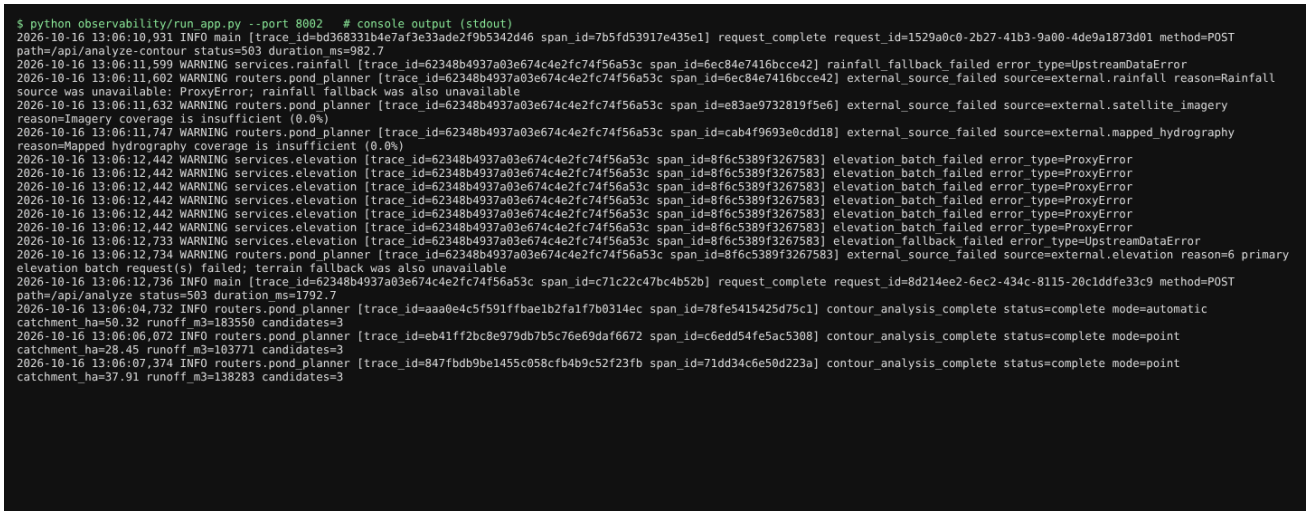


Figure 11: Application console output with trace correlation. All WARN lines of the failing live request share trace id `62348b49...`, the same trace as in the live-failure trace figure, so a developer can jump from a log line straight to the full trace.

4 Understanding of the collected data

Traces answer "what happened in this one request, and where did the time go?"

- The success trace shows external fetches overlapping in time, which proves the `asyncio.gather` concurrency described in my design.
- Pure algorithm time is about 0.9 s of the 1.3–1.5 s request. The rest is reading the 6.7 MB upload and serialising the large JSON response (contours and catchment polygon).
- **A finding from tracing:** there is a gap of about 0.37 s before the external spans start, and `contour.parse_kml` appears again later. The route parses the KML once to get the map centre and the analyser parses it a second time. Passing the already-parsed dataset to the analyser would save about 0.3 s (roughly 25%) per request. I would not have noticed this without the waterfall view.

Metrics answer "how is the service behaving over time?" Request counts by status give the error rate. The duration histograms give p95 latency for a service-level objective, for example "95% of contour analyses under 3 s". The failure counter shows which dependency is unreliable. The catchment and runoff histograms monitor the *scientific output* distribution, so a sudden shift (e.g. all catchments becoming tiny) would reveal a data or code regression even when HTTP status codes are all 200. Metrics are cheap aggregates, so they can be kept for long periods and used for alerts, whereas traces show individual examples.

Logs answer "what exactly was the message?" They hold human-readable detail (failure reasons, result summaries). Because every log carries the trace id, a WARN line in the console leads straight to the full trace in Jaeger, and a slow trace leads back to its logs.

How this helps my application:

- **Monitoring:** alert on a rising 5xx rate or rising `pond.external_source.failures`. My app depends on four public data services, which is exactly where earlier demo failures came from.
- **Debugging:** a 503 trace immediately names the failing source and the number of retries, instead of only "analysis failed".
- **Analysis and optimisation:** stage timings show where to optimise (KML parsing, D8). Rate-limit hits (429) show when the per-minute limit is too strict for a demo.

5 Limitations

- The demonstration ran on my lab machine, not on the allotted college system. That system cannot install packages from PyPI and exposes only one web port, so running the Collector, Jaeger and Prometheus there was not attempted. The code is already in the repository and is switched on with `OTEL_ENABLED=true` plus a Collector endpoint.

- Success-path external data came from local fixtures (Section 2.4), so those three external spans measure fixture time, not network time. The failure-path spans are real network failures.
- Jaeger used in-memory storage and every trace was sampled (100%). Production use would need persistent storage and a sampling policy.
- No Grafana dashboard was built; the Prometheus UI was used for queries.